Intro to Systems Software

Midterm Review

CS/COE 0449

Introduction to Systems Software

Luis Oliveira

(with content borrowed from wilkie and Vinicius Petrucci)

Topics for the Midterm

- Data Representation
  - 2's complement, Floating point, Bitwise operations, Endianness
- C Programming
  - Pointers, Arrays, Strings, Memory Management
- Machine-level Representation (x86-64)
  - Assembly Basics, Control Flow, Calling Conventions, Buffer Overflow
- ~~Executables~~
  - ~~Compilers, Assemblers, Linkers, Loaders~~

Data representation

Encoding Integers

Unsigned

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

Two's Complement

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Sign Bit

- Two's Complement Examples (`w = 5`)

	-16	8	4	2	1	
10 =	0	1	0	1	0	8+2 = 10
-10 =	1	0	1	1	0	-16+4+2 = -10

Unsigned & Signed Numeric Values

X	B2U(*X*)	B2T(*X*)
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	–8
1001	9	–7
1010	10	–6
1011	11	–5
1100	12	–4
1101	13	–3
1110	14	–2
1111	15	–1

- Equivalence
  - Same encodings for nonnegative values
- Uniqueness
  - Every bit pattern represents unique integer value
  - Each representable integer has unique bit encoding
  - Expression containing signed and unsigned `int`:

    `int` is cast to `unsigned`

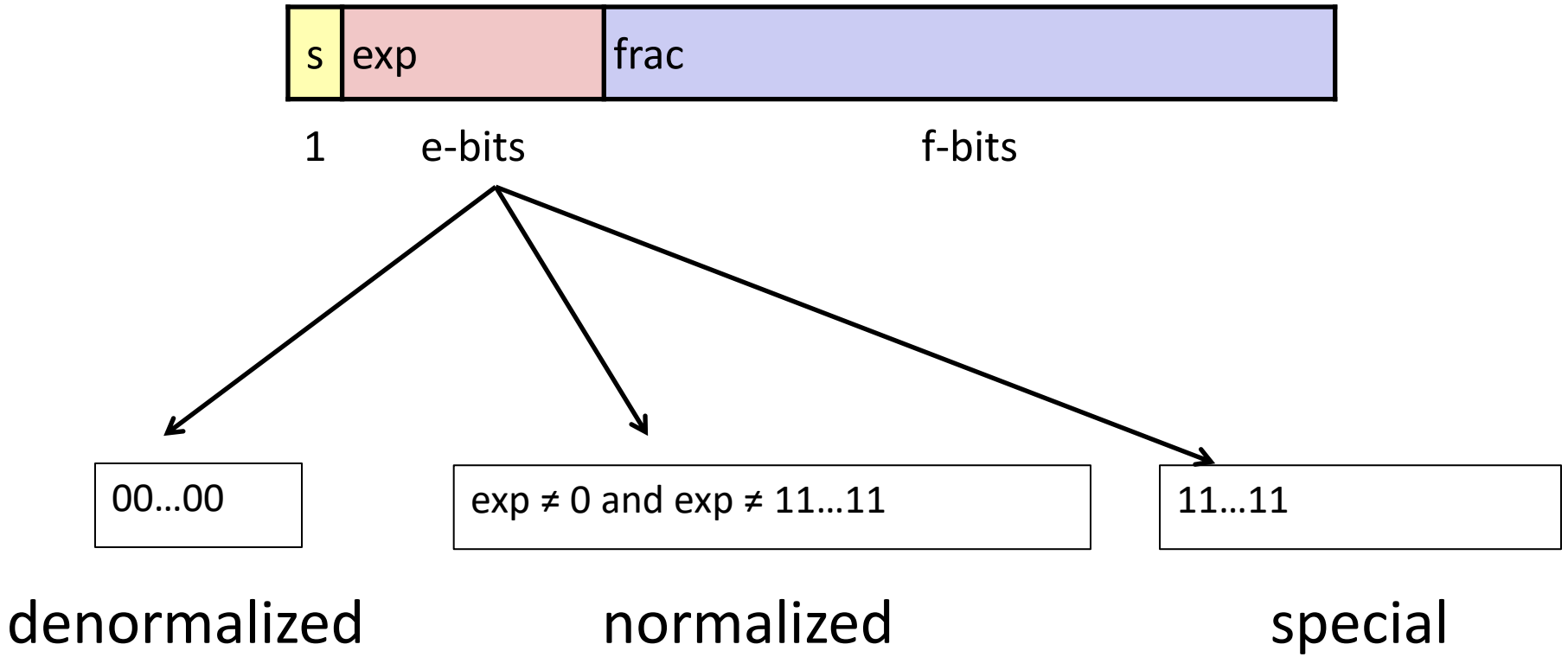
Important: Numeric Ranges

- Unsigned Values
  - UMin = 0
    - 000…0
  - UMax = $2^w - 1$
    - 111…1
- Two's Complement Values
  - TMin = -2^{w-1}
    - 100…0
  - TMax = $2^{w-1} - 1$
    - 011…1
  - Minus 1
    - 111…1

Values for $W = 16$

	Decimal	Hex	Binary
UMax	**65535**	**FF FF**	**11111111 11111111**
TMax	**32767**	**7F FF**	**01111111 11111111**
TMin	**-32768**	**80 00**	**10000000 00000000**
-1	**-1**	**FF FF**	**11111111 11111111**
0	**0**	**00 00**	**00000000 00000000**

Three “kinds” of floating point numbers



"Normalized" Values

- When: **`exp`** ≠ 000…0 and **`exp`** ≠ 111…1

$v = (-1)^s M 2^E$

- Exponent coded as a ***biased*** value: ***E*** = **`exp`** – ***Bias***
  - *exp*: unsigned value of exp field
  - *Bias* = $2^{k-1} - 1$, where *k* is number of exponent bits
    - Single precision: 127 (**`exp`**: 1…254, E: -126…127)
    - Double precision: 1023 (**`exp`**: 1…2046, E: -1022…1023)

- Significand coded with implied leading 1: ***M*** = $1.xxx…x_2$
  - xxx…x: bits of frac field
  - Minimum when **`frac`**=000…0 (M = 1.0)
  - Maximum when **`frac`**=111…1 (M = 2.0 – ε)
  - Get extra leading bit for "free"

Denormalized Values

- Condition: exp = 000…0

$v = (-1)^s M 2^E$
$E = 1 - Bias$

- Exponent value: ***E*** = 1 – Bias (instead of **`exp`** – ***Bias***) (why?)
- Significand coded with implied leading 0: ***M*** = $0.xxx\ldots x_2$
  - **`xxx…x`**: bits of **`frac`**
- Cases
  - **`exp`** = `000…0`, **`frac`** = `000…0`
    - Represents zero value
    - Note distinct values: +0 and –0 (why?)
  - **`exp`** = `000…0`, **`frac`** ≠ `000…0`
    - Numbers closest to 0.0
    - Equispaced



Special Values

- Condition: **`exp`** = **`111…1`**

- Case: **`exp`** = **`111…1`**, **`frac`** = **`000…0`**
  - **Represents value ∞ (infinity)**
  - Operation that overflows
  - Both positive and negative
  - E.g., 1.0/0.0 = −1.0/−0.0 = +∞, 1.0/−0.0 = −∞

- Case: **`exp`** = **`111…1`**, **`frac`** ≠ **`000…0`**
  - **Not-a-Number (NaN)**
  - Represents case when no numeric value can be determined
  - E.g., sqrt(–1), ∞ − ∞, ∞ × 0

Casting floats to ints and vice versa

- `(int) floating_point_expression`
  - Coerces and converts it to the nearest integer (C uses truncation)

```
i = (int) (3.14159 * f);
```

- `(float) integer_expression`
  - Converts integer to nearest floating point

```
f = f + (float) i;
```

float → int → float

```
if (f == (float)((int) f)) {
    printf("true");
}
```

- Does it always print "true"?
  - No, small floating point numbers (< 1) don't have integer representations
- For other numbers, often will be rounding errors

int → float → int

```
if (i == (int)((float) i)) {
  printf("true");
}
```

- Does it always print "true"?
  - No, many large values of integers don't have exact floating point representations
- What about double?
  - Yes, significand is now 52 bits, which can hold all of 32-bit integer, so will always print "true"

Bitwise Operations

- Write a C expression that will yield a word consisting of the least significant byte of `x` and the remaining bytes of `y`
- For operands `x = 0x89ABCDEF` and `y = 0x76543210`, this would give `0x765432EF`

Hint: use masking and bit manipulation

Bitwise Operations

- Write a C expression that will yield a word consisting of the least significant byte of `x` and the remaining bytes of `y`
- For operands `x = 0x89ABCDEF` and `y = 0x76543210`, this would give `0x765432EF`

1) How to extract the upper 3 bytes of x? `(x & 0xFF)`

2) How to extract the lower 1 byte of y? `(y & ~0xFF)`

3) How to combine them? `(x & 0xFF) | (y & ~0xFF)`

Bitwise Operations

- How to perform the operation below using shifts/adders only?

```
a * 7
```

```
(a << 3) - a
```

```
long m12(long x)
{
  return x*12;
}
```

Converted to x86-64 by compiler:

```
leaq (%rdi,%rdi,2), %rax   # t = x+2*x
salq $2, %rax              # return t<<2
```

Bitwise Operations

- Suppose we number the bytes in a *w*-bit word from 0 (least significant) to *w*/8 – 1 (most significant)

- Write code for the following C function, which will return an unsigned value in which byte `i` of argument `x` has been replaced by byte `b`:

```
unsigned replace_byte (unsigned x, int i, unsigned char b);
```

```
replace_byte(0x12345678, 2, 0xAB) --> 0x12AB5678
replace_byte(0x12345678, 0, 0xAB) --> 0x123456AB
```

Bitwise Operations

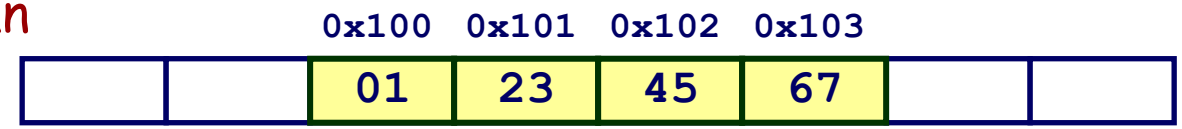
- Write code for the following C function, which will return an unsigned value in which byte `i` of argument `x` has been replaced by byte `b`:

```
unsigned replace_byte (unsigned x, int i, unsigned char b) {
  int itimes8 = i << 3;
  unsigned mask = 0xFF << itimes8;
  return (x & ~mask) | (b << itimes8);
}
```

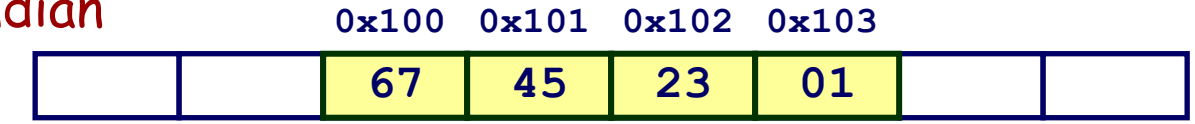
Byte Ordering

- Example
  - Variable `x` has 4-byte value of `0x01234567`
  - Address given by `&x` is `0x100`

Big Endian



Little Endian



C Programming

C Memory Layout Analysis

Consider the following C code:

```c
int a = 5;
void foo() { int temp; }
int main() {
  int b = 0;
  char* s1 = "cs449";
  char s2[] = "cs449";
  char* c = malloc(sizeof(char) * 100);
  foo();
  return 0;
};
```

For each of the following values, state the location in the memory layout where they are stored.

`s1[0]` → ??

`s2[0]` → ??

C Programming

- What is wrong with the C code below?

```
int* pi = malloc(314*sizeof(int));
if(!raspberry) {
    pi = malloc(1 *sizeof(int));
}
return pi;
```

There's a memory leak if raspberry is false as the original value of pi will be unreachable

C programming

Consider the following code snippet which allocates an array and sets the values:

```
1    int main(int argc, char** argv) {
2          int *a = (int*) malloc(213 * sizeof(int));
3          for (int i=0; i<213; i++) {
4                if (a[i] == 0) a[i]=i;
5                else a[i]=-i;
6          }
7          return 0;
8    }
```

Which lines have a problem and how can you fix it?

C programming

```
1    int main(int argc, char** argv) {
2            int *a = (int*) malloc(213 * sizeof(int));
             if (a == NULL) return 0;
3            for (int i=0; i<213; i++) {
4                    if (a[i] == 0) a[i]=i;
5                    else a[i]=-i;
6            }
7            return 0;
8    }
```

malloc can fail!

C programming

```
1    int main(int argc, char** argv) {
2            int *a = (int*) calloc(213, sizeof(int));
             if (a == NULL) return 0;
3            for (int i=0; i<213; i++) {
4                    if (a[i] == 0) a[i]=i;
5                    else a[i]=-i;
6            }
7            return 0;
8    }
```

Allocated memory is not initialized!

C programming

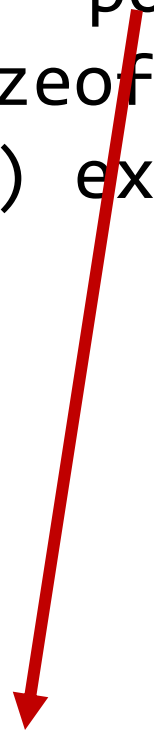
```
1    int main(int argc, char** argv) {
2            int *a = (int*) calloc(213, sizeof(int));
             if (a == NULL) return 0;
3            for (int i=0; i<213; i++) {
4                    if (a[i] == 0) a[i]=i;
5                    else a[i]=-i;
6            }
             free(a);
7            return 0;
8    }
```

All allocated memory must be freed!

C programming

What lines make `safe_int_malloc` not so safe?

```
1   int *safe_int_malloc(int *pointer) {
2       pointer = malloc(sizeof(int));
3       if (pointer == NULL) exit(-1);
4       return &pointer;
5   }
```



`pointer` is a local copy of the pointer! Modifying `*pointer` only changes the value within the scope of this function not outside

C programming

Passing in an `int**` let's us change the value of `int*` pointer

```
1   int *safe_int_malloc(int **pointer) {
2       *pointer = malloc(sizeof(int));
3       if (pointer == NULL) exit(-1);
4       return &pointer;
5   }
```

What's STILL wrong?

C programming

The address of something on the stack will be invalid after the function's execution

```
1   int *safe_int_malloc(int **pointer) {
2       *pointer = malloc(sizeof(int));
3       if (pointer == NULL) exit(-1);
4       return &pointer;
5   }
```



&pointer is a location on the stack in safe_int_malloc's frame!

C programming

```
1   int **safe_int_malloc(int **pointer) {
2       *pointer = malloc(sizeof(int));
3       if (pointer == NULL) exit(-1);
4       return pointer;
5   }
```

Pointers

- Pointer: stores address of some value in memory

- Example:
  - Let us have a pointer `a` where `int* a = 0x100`

    `*a` = accesses value stored at location `0x100`

    `a + i = 0x100 + sizeof(*a) * i`
  - Dereferencing a `NULL` pointer or any other invalid memory address causes **segmentation fault**

x86 programming

x86-64: Stack Manipulation

We execute:

```
mov $0x449, %rax
pushq %rax
```

For each of the following instructions, determine if they will result in the value 0x449 being placed in %rcx?

1) mov (%rsp), %rcx
2) mov 0x8(%rsp), %rcx
3) mov %rsp, %rcx
4) popq %rcx

x86-64: Stack is memory

We execute:

```
mov $0x449, %rax
pushq %rax
popq %rax
```

If we now execute: `mov -0x8(%rsp), %rcx`
what value is in `%rcx`?

1) 0x0 / NULL
2) Seg fault
3) Unknown
4) 0x449

x86-64: Sometimes arguments are implicit

* How many arguments does "rsr" take?
* How many registers are changed before the function call?

(Note, `%sil` is the lower 1 byte of `%rsi`)

```
0x0400596 <+0>:     cmp    %sil,(%rdi,%rdx,1)
0x040059a <+4>:     je     0x4005ae <rsr+24>
0x040059c <+6>:     sub    $0x8,%rsp
0x04005a0 <+10>:    sub    $0x1,%rdx
0x04005a4 <+14>:    callq  0x400596 <rsr>
0x04005a9 <+19>:    add    $0x8,%rsp
0x04005ad <+23>:    retq
0x04005ae <+24>:    mov    %edx,%eax
0x04005b0 <+26>:    retq
```

x86-64: Arguments can already be “correct”

`rsr` does not modify `s` and `t`, so the arguments in those registers are always correct

```
int rsr(char* s, char t, size_t pos)
{
  if (s[pos] == t) return pos;
  return rsr(s, t, pos - 1);
}
```

Example: Reverse-engineer Assembly to C

```
mystery(long, long, long):
  leaq (%rdi,%rsi), %rax
  addq %rdx, %rax
  cmpq $3, %rdi
  jg .L2
  cmpq %rdx, %rsi
  jge .L3
  movq %rdi, %rax
  imulq %rsi, %rax
  ret
.L3:
  movq %rsi, %rax
  imulq %rdx, %rax
  ret
.L2:
  cmpq $10, %rdi
  jle .L1
  movq %rdi, %rax
  imulq %rdx, %rax
.L1:
  ret
```

```
long mystery(long x, long y, long z)
{
      long val = _______________;

      if (__________) {

          if (_________) {

              val = ___________;

          } else {

              val = ___________;
          }

       } else if (___________) {

            val = __________;
       }

       return ________;
}
```

Example: increment

```
long increment(long *p, long val) {
    long x = *p;
    long y = x + val;
    *p = y;
    return x;
}
```

```
increment:
  movq    (%rdi), %rax
  addq    %rax, %rsi
  movq    %rsi, (%rdi)
  ret
```

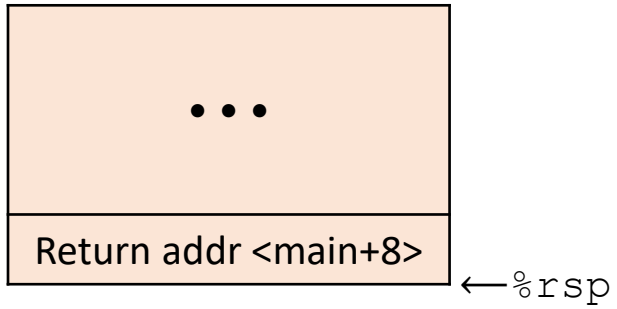
Register	Use(s)
%rdi	1st arg (p)
%rsi	2nd arg (val), y
%rax	x, return value

Procedure Call Example (initial state)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Initial Stack Structure



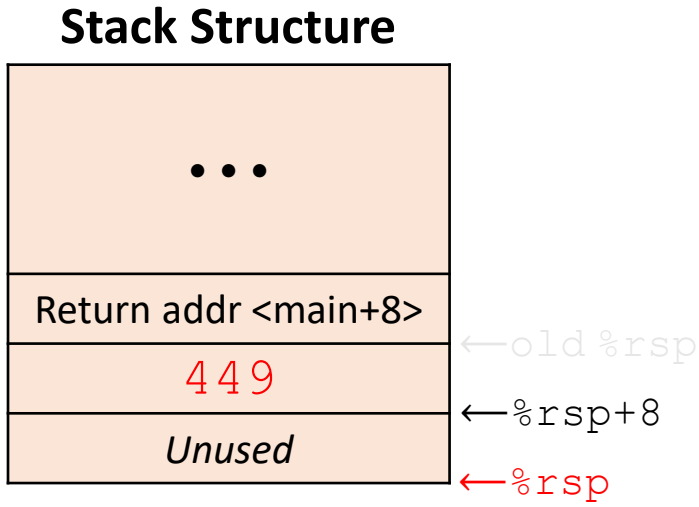
- Return address on stack is the address of instruction immediately *following* the call to "`call_incr`"
  - Shown here as `main`, but could be anything)
  - Pushed onto stack by `call call_incr`

Procedure Call Example (step 1)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Allocate space for local vars



- Setup space for local variables
  - Only `v1` needs space on the stack
- Compiler allocated extra space
  - Often does this for a variety of reasons, including alignment

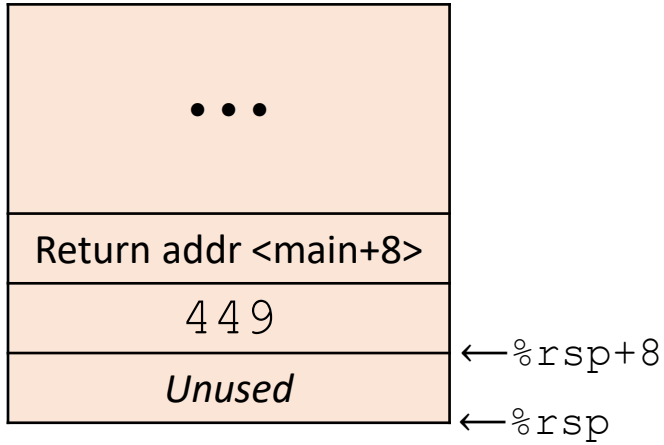
Procedure Call Example (step 2)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Set up parameters for call to `increment`

Stack Structure



Aside: `movl` is used because 100 is a small positive value that fits in 32 bits. High order bits of `rsi` get set to zero automatically. It takes *one less byte* to encode a `movl` than a `movq`.

Register	Use(s)
%rdi	&v1
%rsi	100

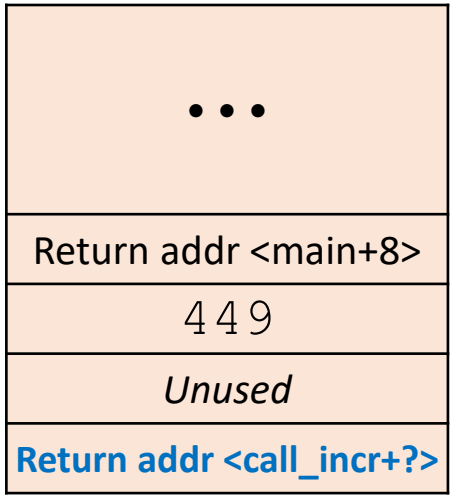
Procedure Call Example (step 3)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

```
increment:
  movq    (%rdi), %rax
  addq    %rax, %rsi
  movq    %rsi, (%rdi)
  ret
```

Stack Structure



- State while inside `increment`
  - **Return address** on top of stack is address of the `addq` instruction immediately following call to `increment`

Register	Use(s)
%rdi	&v1
%rsi	100
%rax	

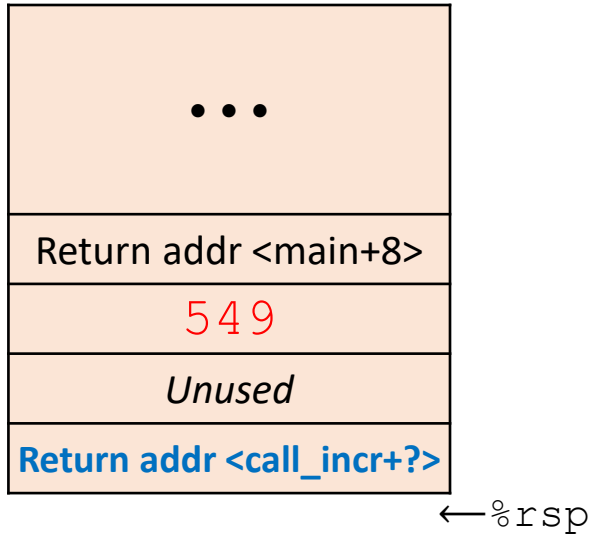
Procedure Call Example (step 4)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

```
increment:
  movq    (%rdi), %rax  # x = *p
  addq    %rax, %rsi    # y = x + 100
  movq    %rsi, (%rdi)  # *p = y
  ret
```

Stack Structure



- State while inside `increment`
  - *After* code in body has been executed

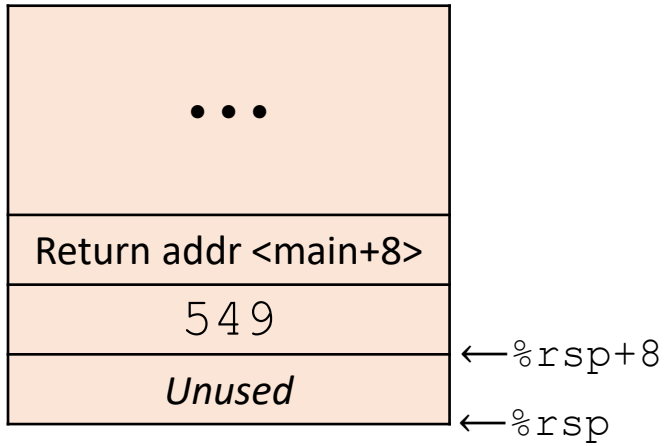
Register	Use(s)
%rdi	&v1
%rsi	549
%rax	449

Procedure Call Example (step 5)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Stack Structure



- After returning from call to `increment`
  - Registers and memory have been modified and return address has been popped off stack

Register	Use(s)
%rdi	&v1
%rsi	549
%rax	449

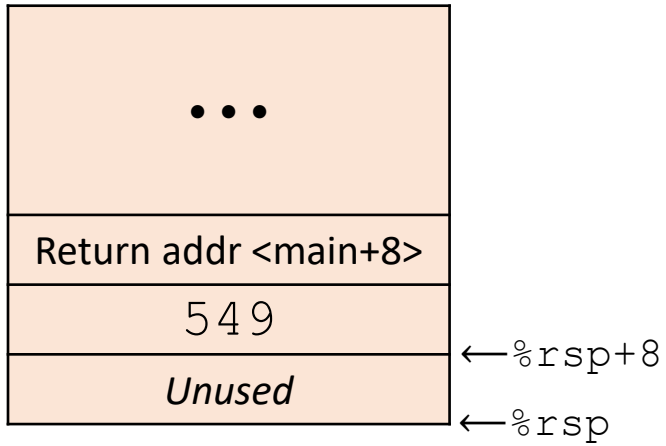
Procedure Call Example (step 6)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Update `%rax` to contain `v1+v2`

Stack Structure



Register	Use(s)
%rdi	&v1
%rsi	549
%rax	549+449

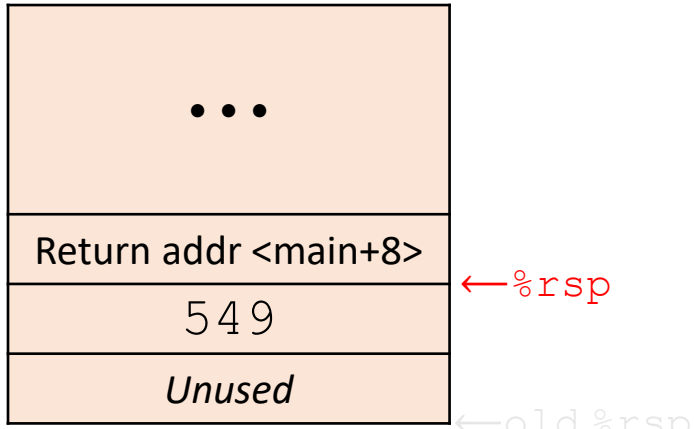
Procedure Call Example (step 7)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

← De-allocate space for local vars

Stack Structure



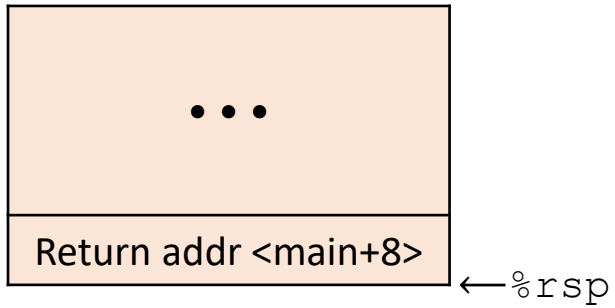
Register	Use(s)
%rdi	&v1
%rsi	549
%rax	998

Procedure Call Example (step 8)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Stack Structure



- State *just before* returning from call to `call_incr`

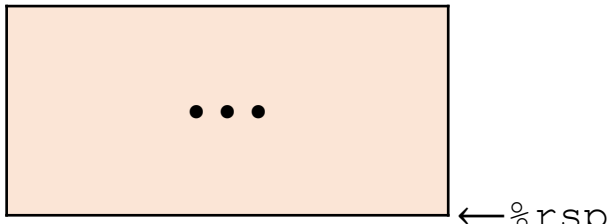
Register	Use(s)
%rdi	&v1
%rsi	549
%rax	998

Procedure Call Example (step 9)

```
long call_incr() {
    long v1 = 449;
    long v2 = increment(&v1, 100);
    return v1 + v2;
}
```

```
call_incr:
  subq    $16, %rsp
  movq    $449, 8(%rsp)
  movl    $100, %esi
  leaq    8(%rsp), %rdi
  call    increment
  addq    8(%rsp), %rax
  addq    $16, %rsp
  ret
```

Final Stack Structure



- State immediately *after* returning from call to `call_incr`
  - Return addr has been popped off stack
  - Control has returned to the instruction immediately following the call to `call_incr` (not shown here)

Register	Use(s)
%rdi	&v1
%rsi	549
%rax	998